



Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

Program design: Introduction

Copyright ©2026 by the 2026 staff of COS 110. All rights reserved.

Practical 4: Smart Home

Release Date: 28-09-2026 at 06:00

Due Date: 02-10-2026 at 23:59

Late Deadline: 03-10-2026 at 00:59

Total Marks: 276

**Read the entire specification before starting
with the practical.**

Contents

1	General Instructions	4
2	Overview	5
2.1	Mark Breakdown	5
3	Your Task:	6
3.1	SmartDevice	6
3.1.1	Variables	6
3.1.1.1	- string deviceID	6
3.1.1.2	- bool isOn	6
3.1.2	Functions	6
3.1.2.1	+ SmartDevice(string id)	6
3.1.2.2	+ SmartDevice* clone() const	6
3.1.2.3	+ ~ SmartDevice()	6
3.1.2.4	+ void executePrimaryFunction()	6
3.1.2.5	+ bool getIsOn() const	6
3.1.2.6	+ void setIsOn(bool status)	6
3.1.2.7	+ string getDeviceID() const	7
3.1.2.8	+ string getDeviceType() const	7
3.1.2.9	+ void printStatus() const	7
3.2	SmartThermostat	8
3.2.1	Variables	8
3.2.1.1	- double currentTemp	8
3.2.1.2	- double targetTemp	8
3.2.2	Functions	8
3.2.2.1	+ SmartThermostat(string id, double targetT)	8
3.2.2.2	+ SmartDevice* clone() const	8
3.2.2.3	+ ~ SmartThermostat()	8
3.2.2.4	+ void executePrimaryFunction()	8
3.2.2.5	+ string getDeviceType() const	8
3.2.2.6	+ void setTemperature(double newTarget)	8
3.2.2.7	+ void setTemperature(double newTarget, bool activateImmediately)	9
3.2.2.8	+ void printStatus() const	9
3.3	SmartLight	10
3.3.1	Variables	10
3.3.1.1	- int brightness	10
3.3.2	Functions	10
3.3.2.1	+ SmartLight(string id, int initialBrightness)	10
3.3.2.2	+ SmartDevice* clone() const	10
3.3.2.3	+ ~ SmartLight()	10
3.3.2.4	+ void executePrimaryFunction()	10

3.3.2.5	+ string getDeviceType() const	10
3.3.2.6	+ void dimLight(int percentage)	10
3.3.2.7	+ void printStatus() const	11
3.4	SmartHomeManager	12
3.4.1	Variables	12
3.4.1.1	- SmartDevice** devices	12
3.4.1.2	- int deviceCount	12
3.4.1.3	- int capacity	12
3.4.2	Functions	12
3.4.2.1	+ SmartHomeManager(int maxCapacity)	12
3.4.2.2	+ ~ SmartHomeManager()	12
3.4.2.3	+ SmartHomeManager(const SmartHomeManager& other)	12
3.4.2.4	+ SmartHomeManager& operator=(const SmartHomeManager& other)	12
3.4.2.5	+ void addDevice(SmartDevice* newDevice)	13
3.4.2.6	+ void runSystemDiagnostic()	13
3.4.2.7	+ void activateAllDevices()	13
3.4.2.8	+ SmartDevice* removeDevice(string id)	13
3.4.2.9	+ int getActiveDeviceCount() const	13
3.4.2.10	+ void deactivateAllDevices()	13
3.5	UML Class Diagram	14
4	Memory Management	14
5	Testing	14
6	Implementation Details	15
7	Upload Checklist	15
8	Submission	16

1 General Instructions

- *Read the entire assignment thoroughly before you begin coding.*
- This assignment should be completed individually.
- Every submission will be inspected with the help of dedicated plagiarism detection software.
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at <https://portal.cs.up.ac.za/files/departmental-guide/>.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

Welcome to the **Smart Home Automation System**! You need to implement a smart home system that will keep track of all the smart devices in your home. This will allow you to switch devices on and off from within your home, on vacation, and even on your way home after going out. **You can assume that all inheritance is public.**

2.1 Mark Breakdown

The following table represents the total marks assigned to each task on FitchFork:

Task	Mark
1 _SMARTDEVICE	14
2 _SMARTLIGHT	24
3 _SMARTTHERMOSTAT	96
4 _SMARTHOMEMANAGER	30
5 _SMARTHOMEMANAGER	37
6 _SMARTHOMEMANAGER	49
7 _TESTING	26
Total	273

3 Your Task:

You are required to implement all of the functions laid out in this section.

3.1 SmartDevice

This class serves as the foundation for your system. It is an abstract class, meaning it cannot be instantiated directly and it forces derived classes to implement the core functionality.

3.1.1 Variables

3.1.1.1 - `string deviceID`

- Unique identifier for the device, e.g. `LIVING_ROOM_T1`.

3.1.1.2 - `bool isOn`

- Tracks whether the device is currently active.

3.1.2 Functions

3.1.2.1 + `SmartDevice(string id)`

- This is the constructor that initialises `deviceID` and sets `isOn` to false by default.

3.1.2.2 + `SmartDevice* clone() const`

- This is a pure virtual function.

3.1.2.3 + `~ SmartDevice()`

- This is a virtual destructor.
- All dynamic memory should be deallocated.

3.1.2.4 + `void executePrimaryFunction()`

- This is a pure virtual function.
- Has no implementation in this class.
- It forces all derived classes to define their specific action.

3.1.2.5 + `bool getIsOn() const`

- Returns the value of `isOn`.

3.1.2.6 + `void setIsOn(bool status)`

- Sets `isOn` to the value of `status`.

3.1.2.7 + string getDeviceID() const

- Returns the value of `deviceId`.

3.1.2.8 + string getDeviceType() const

- This is a virtual function.
- Returns "Generic_Smart_Device\$", where _ represents a space, and \$ represents the end of the line.

3.1.2.9 + void printStatus() const

- This is a virtual function.
- Prints the `deviceId` and whether it is **ON** or **OFF**. It should be in the following format where _ represents a space, \$ represents the end of the line, and everything between { and } represents a variable:

```
Device_ID:_{deviceId}$  
The_device_is_currently_{isOn}$
```

1
2

3.2 SmartThermostat

This class represents a temperature control device. It *is a* SmartDevice.

3.2.1 Variables

3.2.1.1 - double currentTemp

- The current room temperature.

3.2.1.2 - double targetTemp

- The desired temperature setting.

3.2.2 Functions

3.2.2.1 + SmartThermostat(string id, double targetT)

- This constructor passes the `id` to the base constructor and sets the target temperature.
- Set the current temperature to **20**.

3.2.2.2 + SmartDevice* clone() const

- This is a virtual function.
- Subclasses must implement this function to return a deep copy of the calling object.
- Returns a pointer of type `SmartDevice*`.

3.2.2.3 + ~ SmartThermostat()

- This is a virtual destructor.
- All dynamic memory should be deallocated.

3.2.2.4 + void executePrimaryFunction()

- This is a virtual function.
- If `isOn` is true, adjust the `currentTemp` closer to `targetTemp` by 0.5 degrees.

3.2.2.5 + string getDeviceType() const

- Returns "Smart_Thermostat\$", where `_` represents a space, and `$` represents the end of the line.

3.2.2.6 + void setTemperature(double newTarget)

- Set `targetTemp` to be the passed in value.

3.2.2.7 + void setTemperature(double newTarget, bool activateImmediately)

- Set `targetTemp` to be the passed in value if `activateImmediately` is true.
- Else adjust the `targetTemp` closer to the `newTarget` by 0.5 degrees.

3.2.2.8 + void printStatus() const

- This is a virtual function.
- Calls the base class `printStatus` first, then prints the specific thermostat data. It should be in the following format where `_` represents a space, `$` represents the end of the line, and everything between `{` and `}` represents a variable:

```
The_current_temperature:_{currentTemp}$  
The_target_temperature:_{targetTemp}$
```

1
2

3.3 SmartLight

This class represents a dimmable light fixture. It *is a* SmartDevice.

3.3.1 Variables

3.3.1.1 - int brightness

- This scales from 0 to 100.

3.3.2 Functions

3.3.2.1 + SmartLight(string id, int initialBrightness)

- This constructor passes the `id` to the base constructor and sets the brightness level.

3.3.2.2 + SmartDevice* clone() const

- This is a virtual function.
- Subclasses must implement this function to return a deep copy of the calling object.
- Returns a pointer of type `SmartDevice*`.

3.3.2.3 + ~ SmartLight()

- This is a virtual destructor.
- All dynamic memory should be deallocated.

3.3.2.4 + void executePrimaryFunction()

- This is a virtual function.
- If `isOn` is true, emit the light at the designated brightness percentage by printing the message below.
- It should be in the following format where `_` represents a space, `$` represents the end of the line, and everything between `{` and `}` represents a variable:

```
The_light_is_ON_at_{brightness}%$
```

1

3.3.2.5 + string getDeviceType() const

- Returns `"Smart_Light$"`, where `_` represents a space, and `$` represents the end of the line.

3.3.2.6 + void dimLight(int percentage)

- Decreases the `brightness` by the passed in value.
- The `brightness` cannot be less than 0%.

3.3.2.7 + void printStatus() const

- This is a virtual function.
- Calls the base class `printStatus` first, then prints the specific light data. It should be in the following format where `_` represents a space, `$` represents the end of the line, and everything between `{` and `}` represents a variable:

```
The_brightness_of_the_light:_{brightness}$
```

1

3.4 SmartHomeManager

This class handles the collection of devices.

3.4.1 Variables

3.4.1.1 - SmartDevice** devices

- A dynamic array of SmartDevice pointers.
- There will always be gaps at the end of the array, therefore all functions modifying the array should ensure this stays true.

3.4.1.2 - int deviceCount

- Tracks how many devices are currently in the system.

3.4.1.3 - int capacity

- Tracks the maximum allocated size of the array.

3.4.2 Functions

3.4.2.1 + SmartHomeManager(int maxCapacity)

- The constructor that allocates the dynamic array.
- The deviceCount should start at 0.
- If maxCapacity is less than 1, the capacity should be 1.

3.4.2.2 + ~ SmartHomeManager()

- Implements the destructor.

3.4.2.3 + SmartHomeManager(const SmartHomeManager& other)

- This is the copy constructor.
- Must perform a deep copy of the other.devices array and all contained device objects.
- Ensure remaining unused array slots up to capacity are initialised to NULL.

3.4.2.4 + SmartHomeManager& operator=(const SmartHomeManager& other)

- This is the assignment operator.
- Must perform a deep copy of the other.devices array and all contained device objects.
- Return a reference to the current object.

3.4.2.5 + void addDevice(SmartDevice* newDevice)

- This function adds a device pointer to the collection using a shallow copy.
- Only valid devices should be added.

3.4.2.6 + void runSystemDiagnostic()

- Iterates through the entire array and calls `printStatus()` on every device.

3.4.2.7 + void activateAllDevices()

- Iterates through the entire array and switches `isOn` to true, and calls `executePrimaryFunction()` on each item.

3.4.2.8 + SmartDevice* removeDevice(string id)

- Searches the `devices` array for a device matching the given `id` and removes the device.
- Returns the pointer to the removed device.
- If not found, or if the array is empty, returns NULL.

3.4.2.9 + int getActiveDeviceCount() const

- Iterates through the entire array and counts how many devices are active.

3.4.2.10 + void deactivateAllDevices()

- Iterates through the entire array and switches `isOn` to false.

3.5 UML Class Diagram

You are required to submit a complete UML Class Diagram representing the architecture of the Smart Home Automation System. Your diagram must accurately reflect all relationships, classes, member variables, and member functions specified in this document.

Your `draw.io` file should be submitted to the “Practical 4 - Design” slot on the CS Portal.

NB: You will receive a mark of zero if you do not upload a .drawio file.

4 Memory Management

As memory management is a core part of C++ programming, each task on FitchFork will allocate approximately 10% of the marks to memory management. The following command is used:

```
valgrind --leak-check=full ./main
```

1

Please ensure, at all times, that your code *correctly* de-allocates *all* the memory that was allocated.

5 Testing

As testing is a vital skill that all software developers need to know and be able to perform daily, 10% of the assignment marks will be allocated to your testing skills. To do this, you will need to submit a testing main (inside the `main.cpp` file) that will be used to test an Instructor Provided solution. You may add any helper functions to the `main.cpp` file to aid your testing. In order to determine the coverage of your testing the `gcov`¹ tool, specifically the following version `gcov (Debian 8.3.0-6) 8.3.0`, will be used. The following set of commands will be used to run `gcov`:

```
g++ --coverage *.cpp -o main
./main
gcov -f -m -r -j ${files}
```

1

2

3

This will generate output which we will use to determine your testing coverage. The following coverage ratio will be used:

$$\frac{\text{number of lines executed}}{\text{number of source code lines}}$$

We will scale this ration based on class size.

The mark you will receive for the testing coverage task is determined using Table 1:

Coverage ratio range	% of testing mark
0%-5%	0%
5%-20%	20%
20%-40%	40%

¹For more information on `gcov` please see <https://gcc.gnu.org/onlinedocs/gcc/Gcov.html>

40%-60%	60%
60%-80%	80%
80%-100%	100%

Table 1: Mark assignment for testing

Note the top boundary for the Coverage ratio range is not inclusive except for 100%. Also, note that only the functions stipulated in this specification will be considered to determine your testing mark. Remember that your main will be testing the Instructor Provided code and as such, it can only be assumed that the functions outlined in this specification are defined and implemented.

As you will be receiving marks for your testing main, plagiarism detection will also be performed on the testing main.

6 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use `c++98`.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- You may only use the following libraries:
 - `string`
 - `iostream`

7 Upload Checklist

The following `c++` files should be in a zip archive named `uXXXXXXXXX.zip` where `XXXXXXXXX` is your student number:

- `SmartDevice.h`
- `SmartDevice.cpp`
- `SmartThermostat.h`
- `SmartThermostat.cpp`
- `SmartLight.h`
- `SmartLight.cpp`
- `SmartHomeManager.h`

- `SmartHomeManager.cpp`
- `main.cpp`
- Any textfiles used by your `main.cpp`

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

8 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (<https://ff.cs.up.ac.za/>). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

1

and run with the following command:

```
./main
```

1

Remember your `h` file will be overwritten, so ensure you do not alter the provided `h` files.

You have 10 submissions, and your best mark will be your final mark. Upload your archive to the Practical 4 slot on the FitchFork website. If you submit after the deadline but before the late deadline, a 20% mark deduction will be applied. **No submissions after the late deadline will be accepted!**